AI Assisted Coding

Assignment-12.5

Name: A.sai shrehan

HallTicket: 2303A52299

Batch:45

Task Description #1 (Sorting – Merge Sort Implementation)

• Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

• Instructions:

o Prompt AI to create a function merge_sort(arr) that

sorts a list in ascending order.

o Ask AI to include time complexity and space

complexity in the function docstring.

o Verify the generated code with test cases.

• Expected Output:

o A functional Python script implementing Merge Sort with proper documentation.

```python
def merge_sort(arr):
    """Sort and return a new list using merge sort."""
    if len(arr) <= 1:
        return arr[:]

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    merged = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged


if __name__ == '__main__':
    sample = [38, 27, 43, 3, 9, 82, 10]
    print('Original:', sample)
    print('Sorted:  ', merge_sort(sample))
```

```
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:/Users/Sushri/Document
thon.exe c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/assignment_12-5/task1.py
Sorted:   [3, 9, 10, 27, 38, 43, 82]
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding>
```

Task Description #2 (Searching – Binary Search with AI Optimization)

• Task: Use AI to create a binary search function that finds a target element in a sorted list.

• Instructions:

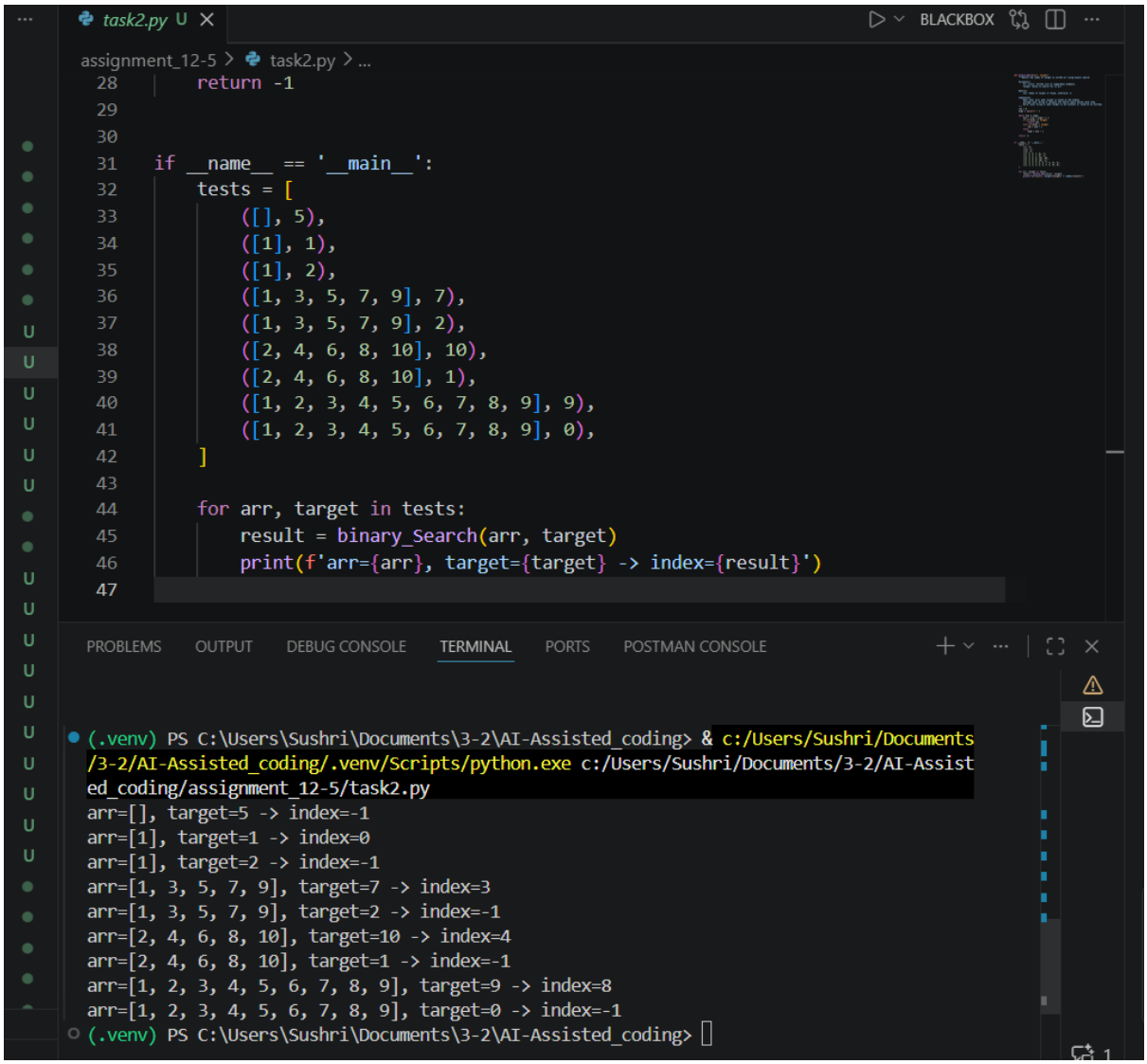
o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.

o Include docstrings explaining best, average, and worst-case complexities.

o Test with various inputs.

• Expected Output:

o Python code implementing binary search with AI-generated comments and docstrings.



Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.

2. Sort appointments based on time or consultation fee.

Student Task

• Use AI to recommend suitable searching and sorting

algorithms.

• Justify the selected algorithms.

• Implement the algorithms in Python.

assignment_12-5 > task3.py > ...

```python
# Task Description #3: Smart Healthcare Appointment Scheduling System
# A healthcare platform maintains appointment records containing
# appointment ID, patient name, doctor name, appointment time, and
# consultation fee. The system needs to:
# 1. Search appointments using appointment ID.
# 2. Sort appointments based on time or consultation fee.
# recommend suitable searching and sorting algorithms.
# • Implement the algorithms in Python.
def search_appointment(appointments, appointment_id):
    for appointment in appointments:
        if appointment['appointment_id'] == appointment_id:
            return appointment
    return None
def sort_appointments_by_time(appointments):
    return sorted(appointments, key=lambda x: x['appointment_time'])

def sort_appointments_by_fee(appointments):
    return sorted(appointments, key=lambda x: x['consultation_fee'])
# Example usage
appointments = [
    {'appointment_id': 'A001', 'patient_name': 'John Doe', 'doctor_name': 'Dr. Smith'
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:/Users/Sushri/Documents
/3-2/AI-Assisted_coding/.venv/Scripts/python.exe c:/Users/Sushri/Documents/3-2/AI-Assist
ed_coding/assignment_12-5/task2.py
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:/Users/Sushri/Documents/3-2/A
I-Assisted_coding/.venv/Scripts/python.exe c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/as
signment_12-5/task3.py
Appointment found: {'appointment_id': 'A001', 'patient_name': 'John Doe', 'doctor_name': 'Dr.
Smith', 'appointment_time': '2024-07-01 10:00', 'consultation_fee': 100}
Appointments sorted by time:
{'appointment_id': 'A001', 'patient_name': 'John Doe', 'doctor_name': 'Dr. Smith', 'appointmen
t_time': '2024-07-01 10:00', 'consultation_fee': 100}
Appointments sorted by consultation fee:
{'appointment_id': 'A001', 'patient_name': 'John Doe', 'doctor_name': 'Dr. Smith', 'appointmen
t_time': '2024-07-01 10:00', 'consultation_fee': 100}
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding>
```

Task Description #4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Student Task

- Identify efficient algorithms using AI assistance.
- Justify the algorithm choices.
- Implement searching and sorting in Python.

task4.py U ×

assignment_12-5 > task4.py > ...

```python
from dataclasses import dataclass
from datetime import datetime
from typing import List, Optional


@dataclass
class Booking:
    ticket_id: str
    passenger_name: str
    train_number: str
    seat_number: str
    travel_date: str

    def travel_date_obj(self) -> datetime:
        return datetime.strptime(self.travel_date, '%Y-%m-%d')


def search_ticket_by_id(bookings: List[Booking], ticket_id: str) -> Optional[Booking]:
    """Return booking matching ticket_id, or None if not found."""
    for booking in bookings:
        if booking.ticket_id == ticket_id:
            return booking
    return None


def sort_bookings(bookings: List[Booking], key: str) -> List[Booking]:
    """Return a new list of bookings sorted by travelDate or seat_number."""
    if key == 'travelDate':
        return sorted(bookings, key=lambda b: b.travel_date_obj())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:/Users/Sushri/Documents/3-2/A
I-Assisted_coding/.venv/Scripts/python.exe c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/as
signment_12-5/task4.py

Sort by seat_number:
Booking(ticket_id='T003', passenger_name='Charlie', train_number='123A', seat_number='05', tra
vel_date='2026-06-01')
Booking(ticket_id='T002', passenger_name='Bob', train_number='124B', seat_number='08', travel_
date='2026-05-18')
Booking(ticket_id='T001', passenger_name='Alice', train_number='123A', seat_number='12', trave
l_date='2026-05-20')
Booking(ticket_id='T004', passenger_name='Dana', train_number='125C', seat_number='15', travel
_date='2026-05-18')
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding>
```

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.

2. Sort records based on room number or allocation date.

Student Task

• Use AI to suggest optimized algorithms.

• Justify the selections.

• Implement the solution in Python.

assignment_12-5 > task5.py > ...

```python
from dataclasses import dataclass
from datetime import datetime
from typing import List, Optional


@dataclass
class Allocation:
    student_id: str
    room_number: str
    floor: int
    allocation_date: str

    def allocation_date_obj(self) -> datetime:
        return datetime.strptime(self.allocation_date, '%Y-%m-%d')


def search_allocation_by_student_id(allocations: List[Allocation], student_id: str) -> Optional[Allocation]:
    """Search allocation details by student ID.

    Best optimized approach: use a dictionary mapping student IDs to allocations for O(1) lookup.
    If the input list is unsorted, a linear scan is O(n).
    If the list is pre-sorted by student_id, binary search is O(log n).
    """
    for allocation in allocations:
        if allocation.student_id == student_id:
            return allocation
    return None
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/.venv/Scripts/pyth
c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/assignment_12-5/task5.py
Sort by room_number:
Allocation(student_id='S001', room_number='101A', floor=1, allocation_date='2026-07-01')
Allocation(student_id='S002', room_number='102B', floor=1, allocation_date='2026-06-28')
Allocation(student_id='S004', room_number='103C', floor=1, allocation_date='2026-06-30')
Allocation(student_id='S003', room_number='201A', floor=2, allocation_date='2026-07-03')

Sort by allocation_date:
Allocation(student_id='S002', room_number='102B', floor=1, allocation_date='2026-06-28')
Allocation(student_id='S004', room_number='103C', floor=1, allocation_date='2026-06-30')
Allocation(student_id='S001', room_number='101A', floor=1, allocation_date='2026-07-01')
Allocation(student_id='S003', room_number='201A', floor=2, allocation_date='2026-07-03')
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding>
```

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title,

genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.

2. Sort movies based on rating or release year.

Student Task

- Recommend searching and sorting algorithms using AI.
- Justify the chosen algorithms.
- Implement Python functions.

assignment_12-5 > task6.py > ...

```python
from dataclasses import dataclass
from typing import List, Optional


@dataclass
class Movie:
    movie_id: str
    title: str
    genre: str
    rating: float
    release_year: int


def search_movie_by_id(movies: List[Movie], movie_id: str) -> Optional[Movie]:
    """Return the movie with the given movie_id, or None if not found."""
    for movie in movies:
        if movie.movie_id == movie_id:
            return movie
    return None


def sort_movies(movies: List[Movie], key: str) -> List[Movie]:
    """Return a new list of movies sorted by rating or release_year."""
    if key == 'rating':
        return sorted(movies, key=lambda movie: movie.rating, reverse=True)
    if key == 'release_year':
        return sorted(movies, key=lambda movie: movie.release_year)
    raise ValueError("Sort key must be 'rating' or 'release_year'")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding> & c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/.venv/Scripts/pythe
c:/Users/Sushri/Documents/3-2/AI-Assisted_coding/assignment_12-5/task6.py
Sort by rating:
Movie(movie_id='M003', title='Pulse', genre='Thriller', rating=9.2, release_year=2025)
Movie(movie_id='M001', title='Epoch', genre='Sci-Fi', rating=8.9, release_year=2024)
Movie(movie_id='M004', title='Luna', genre='Fantasy', rating=8.0, release_year=2023)
Movie(movie_id='M002', title='Aster', genre='Drama', rating=7.4, release_year=2021)

Sort by release_year:
Movie(movie_id='M002', title='Aster', genre='Drama', rating=7.4, release_year=2021)
Movie(movie_id='M004', title='Luna', genre='Fantasy', rating=8.0, release_year=2023)
Movie(movie_id='M001', title='Epoch', genre='Sci-Fi', rating=8.9, release_year=2024)
Movie(movie_id='M003', title='Pulse', genre='Thriller', rating=9.2, release_year=2025)
(.venv) PS C:\Users\Sushri\Documents\3-2\AI-Assisted_coding>
```